

BALANCEBITE

Syeda Fizza Batool Zaidi

02-136212-043

Bisma Hashmi

02-136212-005

Aleena Ahmed

02-136212-002

A project report submitted in partial fulfilment of the
Requirements for the award of the degree of
Bachelor of Science in (Artificial Intelligence)

Department of Computer Science
Bahria University, Karachi

2025

DECLARATION

We hereby declare that this project report is based on our original work except for citations and quotations which have been duly acknowledged. We also declare that it has not been previously and concurrently submitted for any other degree or award at Bahria University or other institutions.

Signature : _____

Name : Syeda Fizza Batool Zaidi

Reg No : 02-136212-043

Signature : _____

Name : Bisma Hashmi

Reg No : 02-136212-005

Signature : _____

Name : Aleena Ahmed

Reg No : 02-136212-002

Date : 05 – May – 2025

APPROVAL FOR SUBMISSION

We certify that this project report entitled **"BALANCEBITE"** was prepared by **SYEDA FIZZA BATOOL ZAIDI, BISMA HASHMI, ALEENA AHMED** has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor of Science in **Artificial Intelligence** at Bahria University, Karachi.

Approved by,

Signature : _____

Supervisor : Dr Taha Jilani

Date : _____

The copyright of this report belongs to Bahria University according to the Intellectual Property Policy of Bahria University BUORIC-P15 amended on April 2019. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

© 2025 Bahria University. All right reserved.

ACKNOWLEDGEMENTS

We would like to thank everyone who had contributed to the successful completion of this project. We would like to express my gratitude to my project supervisor, Dr. Taha Jilani for his invaluable advice, guidance and his enormous patience throughout the development of the research.

In addition, we would also like to express our gratitude to our loving parents and friends who have helped and given us encouragement.

BALANCEBITE

ABSTRACT

Worldwide obesity preys on 50 percent of Pakistani population who carry excessive body weight. The increased rate of obesity creates additional threats to Pakistan's population for the development of chronic illnesses including heart disease and diabetes. The fast-expanding mobile phone market in Pakistan with its 192 million active users creates an optimal environment to fight obesity using creative digital solutions.

BalanceBite represents an AI-powered mobile application which monitors dietary intake and exercise habits of users using sensors and machine learning algorithms [Vision Transformer] to process images of food and evaluate caloric values while measuring bodily movements through sensor analysis for caloric expenditure. The project selects South-Asian cuisine as its main focus because it wants to connect with its local cultural background.

The application creates interactive charts and offers tailore It’s smart, simple and user-friendly. Through its features BalanceBite allows users to implement health-enhancing behaviors while reducing health risks that stem from obesity and improving life quality.

TABLE OF CONTENTS

DECLARATION **ii**

APPROVAL FOR SUBMISSION **iii**

ACKNOWLEDGEMENTS **v**

ABSTRACT **vi**

TABLE OF CONTENTS **vii**

LIST OF TABLES **ix**

LIST OF FIGURES **x**

LIST OF SYMBOLS / ABBREVIATIONS **xi**

LIST OF APPENDICES **xii**

CHAPTER

1 INTRODUCTION **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Aims and Objectives 1

1.4 Scope of Project 2

2 LITERATURE REVIEW **3**

2.1 Obesity and Health Implications 3

2.2 Role of Digital Health Technologies in Obesity Management 3

2.3 Cultural Relevance in Health Interventions 4

2.4 Importance of User-Friendly Features 4

2.4.1 Addressing Obesity with BalanceBite 4

3 REQUIREMENT SPECIFICATIONS **5**

3.1 System Overview and Requirements 5

3.2 Existing System 5

3.2.1 MyFitnessPal 5

3.2.2 Foodvisor 6

3.2.3 HealthifyMe 7

3.2.4 CalorieMama AI 8

	3.3	Limitations of Existing Systems	9
	3.4	Proposed Systems	10
	3.5	Requirement Specifications	10
		3.5.1 Functional Requirements	10
		3.5.2 Personalized Recommendations:	11
	3.6	Non-Functional Requirements	11
4	**DESIGN AND METHODOLOGY**		**13**
	4.1	System Design	13
	4.2	Dataset Preparation	15
		4.2.1 Dataset Creation	15
	4.3	Dataset Preprocessing	16
	4.4	Dataset Labelling	16
5	**SYSTEM IMPLEMENTATION**		**17**
	5.1	Architecture Design	17
	5.2	Frontend Implementation	17
	5.3	Backend Implementation	17
	5.4	Vision Transformer Model	18
6	**SYSTEM TESTING AND EVALUATION**		**19**
	6.1	Testing Methodology	19
	6.2	Performance Evaluation	19
	6.3	Usability Testing	20
	6.4	Accuracy and Reliability	20
7	**CONCLUSION**		**21**
REFERENCES			**22**
APPENDICES			**23**

LIST OF TABLES

TABLE	TITLE	PAGE
Table 1.1	Nutritional Components	18
Table 1.2	Models Comparison	19
Table 2.1	Food Recognition Methods Comparison	22
Table 2.2	Activity Recognition Apps Comparison	24
Table 2.3	Dietary Plan Suggestion Comparison	26
Table 4.1	Comparison Table	36
Table 5.1	Vision Transformer Table	40

LIST OF FIGURES

FIGURE	TITLE	PAGE
Figure 1.1	Scale Of The Obesity Problem	14
Figure 1.2	Interface of Foodvisor	16
Figure 1.3	Interface of Healthifyme	17
Figure 2.1	Interface Of Food Recogntion	21
Figure 2.2	Interface Of Activity Recogntion	23
Figure 2.3	Interface Of Dietary Plans	25
Figure 3.1	Interface of myfitnesspal	27
Figure 3.2	Interface of CaloriemamaAI	28
Figure 4.1	Design and Methodology	33
Figure 4.2	Process of Food identification and nutrition estimation	33
Figure 4.3	Activity identification and calorie estimation process	34
Figure 4.4	Dataset	35
Figure 4.5:	Dataset and Model Training Flowchart	38

LIST OF SYMBOLS / ABBREVIATIONS

ViT	Vision Transformer (AI model used for dish recognition and nutritional analysis)
AI	Artificial Intelligence (Technologies used for personalization and decision-making)
App	Application (The mobile or software platform developed for users)
UI/UX	User Interface/User Experience (Design aspects of the app for ease of use and interaction)
Dataset	A collection of data (used for training the model, includes images and nutritional info)
Region	South Asia (Target geographical location for the application's use)
HCI	Human-Computer Interaction (Study of how users interact with the app)
TFLite	TensorFlow Lite (Optimized version of the model for mobile devices)
ONNX	Open Neural Network Exchange (For cross-platform compatibility in model deployment)
ML	Machine Learning (Broader field encompassing ViT model)

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
APPENDIX A:	Approval Certificate	49

INTRODUCTION

1.1. Background

One of the 21st century's most urgent global public health issues is obesity, which is becoming more widely acknowledged. Obesity is characterized by abnormal or excessive fat buildup that poses a health concern, according to the World Health Organization (WHO). With at least 2.8 million deaths annually attributed to overweight or obesity, obesity has reached epidemic proportions worldwide [1]. Obesity, often thought to be a problem in high-income nations, is now common in low- and middle-income nations as well. Global obesity rates have almost tripled since 1975, according to WHO data [1]. Increased consumption of high-calorie and low-nutrient foods, sedentary lifestyles, urbanization, and technology improvements that encourage physical inactivity are some of the reasons contributing to the worldwide obesity epidemic's acceleration in recent decades.

The situation is equally worrying in Pakistan. According to studies, juvenile obesity is increasing and over half of adults are overweight or obese. Due to the higher risk of related metabolic illnesses in this population, which frequently manifest at lower body mass index (BMI) thresholds than in Western populations, the rise in obesity rates in South Asian nations is especially worrisome [2].

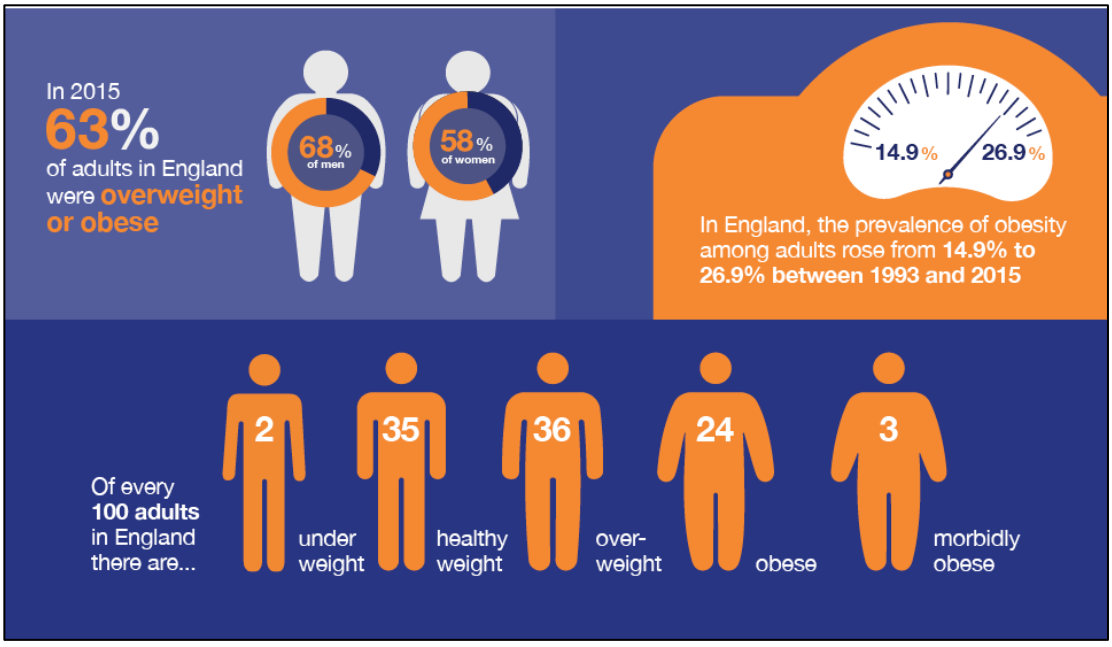


Figure 1.1 Scale Of The Obesity Problem

In 2015 to 2016, 19.8% of children aged 10 to 11 were obese and a further 14.3% were overweight. Of children aged 4 to 5, 9.3% were obese and another 12.8% were overweight. This means a third of 10 to 11 year olds and over a fifth of 4 to 5 year olds were overweight or obese.

1.1.1. Obesity Without Associated Diseases

Although obesity is frequently linked to disorders including type 2 diabetes, heart disease, high blood pressure, and some types of cancer, it can also occur on its own. This condition, which is also known as "metabolically healthy obesity," refers to people who, although having a BMI that makes them fat, do not have the metabolic risk factors that are commonly linked to obesity [3]. Such people may eventually suffer from poor health outcomes, such as joint issues, exhaustion, decreased mobility, and a lower general quality of life, even in the absence of these conditions.

Furthermore, it is impossible to undervalue the psychological effects of obesity. According to [4], people who are obese frequently experience mental health problems such anxiety and depression, low self-esteem, and social stigmatization, even in the absence of concomitant disorders. This emphasizes how critical it is to acknowledge obesity as a multifaceted illness that impacts several aspects of health and wellbeing.

1.1.2. Challenges

1.1.2.1. Cultural Eating Customs:

Traditional South Asian meals are frequently high in lipids and carbohydrates. Overindulgence in calorie-dense foods is common during social and celebratory occasions.

1.1.2.2. Lack of Nutritional Awareness:

According to the [5], a sizable section of the populace is ignorant of fundamental nutritional concepts, such as the calorie and nutrient contents of common foods.

1.1.2.3. Limited Physical Activity:

According to [6], sedentary work settings, a dearth of recreational activities, and urban congestion limit opportunities for physical activity.

1.1.2.4. Barriers to Digital Literacy:

Many people believe that health-related mobile applications are too complex or irrelevant to their lifestyle, according to [7]. The cultural needs of the South Asian population are frequently not met by the several mobile apps that are currently available, such as MyFitnessPal, Foodvisor, and HealthifyMe. The majority of them concentrate on Western eating patterns and offer general, one-size-fits-all guidance that is ineffective for people with diverse dietary and cultural preferences. For instance, physical food entry is necessary for CalorieMama AI and MyFitnessPal, which can be time-consuming and result in erroneous tracking of typical meals. In regions like South Asia, where cuisines deviate significantly from the average Western diet, applications such as Foodvisor are less effective because they do not place enough emphasis on culturally relevant South Asian meals, even though they employ AI to scan and estimate food calories. Additionally, a lot of these applications don't take into account regional cuisines or provide tailored recommendations depending on dietary customs. The adoption and long-term viability of these digital health technologies in South Asian populations may be hampered by this distance.

1.1.2.4.1. Foodvisor

Foodvisor leverages AI to scan pictures and detect food items then determines calorie and nutrient facts to ease meal logging process. It poses substantial competition to BalanceBite's innovation because of this feature [8].

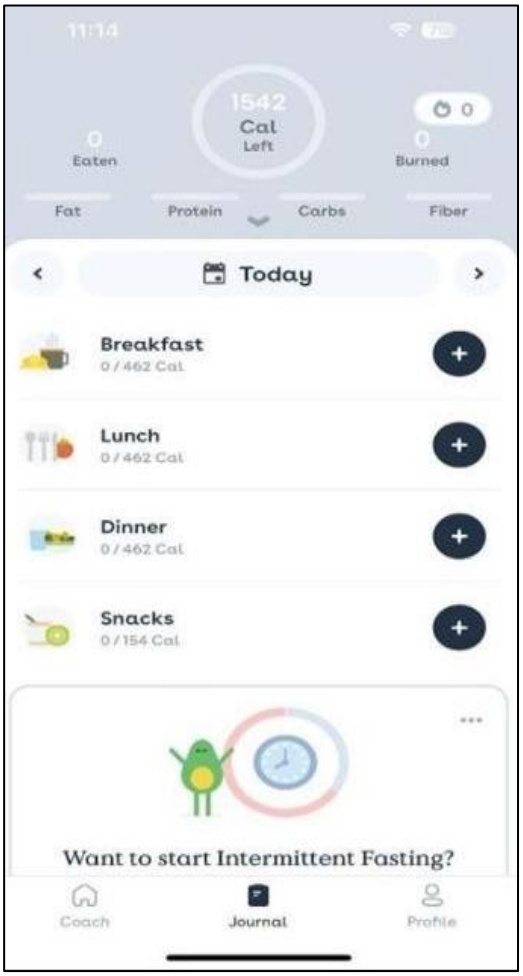


Figure 1.2 Interface of Foodvisor

1.1.2.4.2. HealthifyMe

BalanceBite aims to achieve similar goals as HealthifyMe for Indian users but it does not have advanced AI recognition systems or provides complete South Asian food options [9].



Figure 1.3 Interface of Healthifyme

1.1.2.5. Psychological and Social Stigma:

People who are obese frequently experience prejudice and unfavorable preconceptions, which deters them from getting treatment or altering their lifestyle [4].

1.1.3. Nutritional Components Overview

Addressing obesity requires an understanding of the elements of a balanced diet. The two primary categories of nutrients are macronutrients and micronutrients, and both are essential for preserving health and avoiding weight gain.

Type	Examples	Purpose
Macronutrients	Carbohydrates, Proteins, Fats	Provide energy and structural components for the body
Micronutrients	Vitamins (A, C, D), Minerals (Iron, Calcium, Zinc)	Support immune function, bone health, and metabolic roles
Other Nutrients	Fiber, Water	Aid digestion, hydration, and overall body function

Table 1.1 Nutritional Components

Preventing and controlling obesity requires a balanced consumption of these elements, especially when considering portion sizes and dietary density [1].

1.2. Problem Statement

Even while health and fitness applications are becoming more and more popular, the majority of current solutions are ineffective at catering to South Asian customers. These apps, which offer general guidance and calorie-counting systems that ignore ethnic food diversity and traditional meal arrangements, are frequently created with Western eating habits in mind. Because of this, South Asian users encounter difficulties with manual food entry, inaccurate tracking of their traditional meals, and a dearth of assistance that is suitable for their culture.

In addition to decreasing user engagement, this disparity fuels the region's ongoing rise in obesity and other health problems. A system that automates food detection, takes into account South Asian eating habits, and provides individualized activity and nutrition recommendations is desperately needed. The BalanceBite app fills this gap by providing South Asian consumers with an intelligent, culturally relevant health-tracking experience through the use of smartphone sensors and AI-based picture recognition.

1.3. Aims and Objectives

1.3.1. South-Asian Food Database Development:

The first objective is to create a thorough and culturally appropriate food database with a special emphasis on South Asian cuisines. Comprehensive calorie counts, macronutrient breakdowns (proteins, fats, and carbs), and typical components used in local recipes are all included in this database. It is intended to fill the significant void in culturally inclusive dietary data that can be found in the majority of current nutrition and health applications, which often concentrate on Western diets. Over 300 different South Asian foods were included in the hand curated dataset, which was checked for authenticity and nutritional accuracy by a certified nutritionist. To avoid false recognition and make sure the algorithm can detect when a dish is empty or does not contain legitimate food items, a supplementary dataset was also developed for non-food objects.

1.3.2. AI-Based Food Image Recognition:

The system incorporates cutting-edge AI-based food image identification technology to improve automation and lessen need on human input. To reliably identify the dish type and its nutritional and caloric content, models such as ResNet50 [30], EfficientNet [10], VGG16 [11], and Vision Transformer (ViT) [12] are used to analyze user-submitted food photos. The Vision Transformer model outperformed the others in picture classification tasks on our dataset, hence it was chosen for final deployment. In order to increase model accuracy, the system has also been validated for edge cases utilizing non-food products.

Flutter is used to integrate the recognition capability in a cross-platform mobile application, guaranteeing real-time categorization and an engaging user interface. With less human labor and a lower chance of input errors, this image recognition feature enables the app to provide tailored health advice.

Model	Top-1 Accuracy	Dataset(s)	Handles Imbalanced Data	Key Insights	Source
ResNet50 [30]	87.42%	ETH Food-101	Moderate (with augmentation)	Demonstrated strong performance in food classification tasks.	Enhancing Food Image Recognition by Multi-Level Fusion and the Attention Mechanism
EfficientNet [10]	77.26%	Indonesian Food Dataset	Good (with balancing)	Balanced accuracy with efficiency; suitable for mobile applications.	Performance comparison of state-of-the-art deep learning model architectures in Indonesian food image classification
VGG16 [11]	78.05%	Indonesian Food Dataset	Poor (struggles w/ imbalance)	High parameter count leads to longer inference times; less efficient for real-time applications.	Performance comparison of state-of-the-art deep learning model architectures in Indonesian food image classification
ViT (Vision Transformer) [12]	92.59%	Food101	Excellent (self-attention based)	Achieved state-of-the-art performance, especially when combined with CNNs for local feature extraction.	An Explainable CNN and Vision Transformer-Based Approach for Real-Time Food Recognition

Table 1.2 Models Comparison

1.3.3. Physical Activity Detection via Smartphone Sensors:

In order to accurately estimate calorie expenditure based on movement and physical routines, the project integrates smartphone sensors like accelerometers and gyroscopes to detect user activity.

1.3.4. Personalized Health and Diet Plans:

In order to promote long-term involvement and improved health outcomes, user data will be examined to develop customized fitness regimens and dietary recommendations based on cultural eating habits and individual health objectives.

1.4. Scope of Project

BalanceBite serves the South Asian population through its crucial features that include:

1.4.1. The use of artificial intelligence allows users to determine their food calories and nutrients by taking photographic images.

1.4.2. Smartphone sensors enable activity tracking that allows users to monitor their physical activities to calculate their burned calories.

1.4.3. Cultural Relevance: Focusing on traditional South-Asian dishes for more accurate dietary recommendations.

1.4.4. Healthcare solutions providing adapted healthcare guidance to users through visual graphs together with dietary plans.

1.4.5. The application features a design which makes it accessible to all users no matter their technical skill level.

LITERATURE REVIEW

2.1 Food Recognition

Because food recognition technologies allow users to track their food intake with high precision, they have become quite popular in the management of obesity. Apps may recognize items from user-taken photos, assess their nutritional worth, and offer pertinent dietary advice by utilizing AI-powered image recognition. In South Asian communities, where traditional foods could be misrepresented or not sufficiently included in generic diet regimens, this technique is quite helpful.

By recognizing ingredients, quantity sizes, and possible dietary deficits, food recognition also enables users to learn more about their eating patterns. By increasing consumers' awareness of their dietary intake, this enhances adherence to health plans.

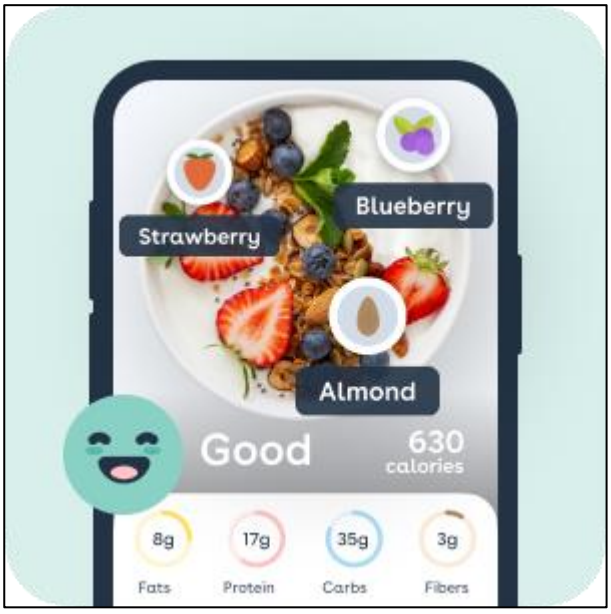


Figure 2.1 Interface Of Food Recogntion

After analyzing a photograph of a meal, the system recognizes the food items and superimposes their labels and calorie counts on the picture.

Feature	**CalorieMama AI** [13]	**HealthifyMe** [9]	**Lose It!** [13]	**Foodvisor** [8]	**BalanceBite**
Sensors Used	Camera (for image recognition), Accelerometer (optional for activity tracking)	Accelerometer, GPS	Barcode Scanner, Snap It, Accelerometer, GPS	Barcode Scanner, Snap It , Accelerometer, GPS	Camera, Accelerometer, Gyroscope, Ambient Light Sensor
Classifiers Used	Convolutional Neural Networks (CNN) for image classification	Deep Learning Models	Barcode scanner classifier, food item matching from database	Deep Learning Models	Convolutional Neural Networks (CNN) for food image classification
What does it identify?	Identifies food types, portion sizes, and nutrients from images using AI-based food recognition	Manually identifies food items from database based on user selection	Identifies packaged foods using barcodes, matching with database entries	Identifies food types, portion sizes, and nutrients from images using AI	Identifies food types, portion sizes, and nutrients from images using AI; user can select quantity or manually enter it
Sensors for Activity Tracking	Syncs with external sensors like Apple Watch, Google Fit for step counting, sleep tracking	Accelerometer, GPS Fit for tracking steps and activities	Syncs with Apple Health, Google Fit for step counting, physical activity	Syncs with Apple Health, Google Fit for activity tracking	Uses only Phone sensors: Accelerometer and Gyroscope (Angular) for activity tracking
Cultural Relevance	Global	India-focused	Global	Global	South-Asian food focus
AI-Powered Food Recognition	Yes	Yes	No	Yes	Yes
Personalized Health Recommendations	Yes	Yes	Yes	Yes	Yes
Focus on Calorie Tracking	Yes	Yes	Yes	Yes	Yes
Historical Record	Yes	Yes	Yes	Yes	Yes

Table 2.1 Food Recognition Methods Comparison

2.2 Activity Recognition

Another crucial component of digital health tools that help people manage their weight is activity recognition. Apps may track and record a variety of physical activities, including walking, running, and cycling, by utilizing smartphone sensors like accelerometers and gyroscopes. Real-time feedback on the level of physical activity is provided by these sensors, which also measure movement patterns and calculate caloric expenditure.

Activities are frequently categorized using AI models, which helps users evaluate their level of fitness and enhance their workout regimens. In order to encourage a more active lifestyle, which is essential in the fight against obesity, activity recognition can be especially helpful.

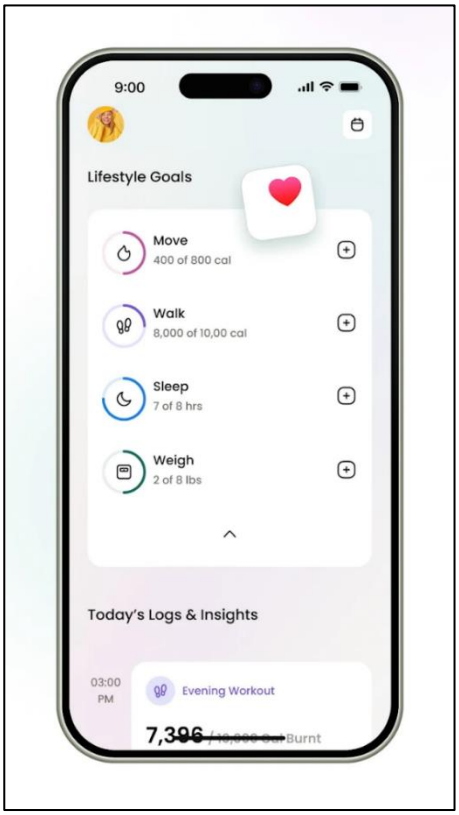


Figure 2.2 Interface Of Activity Recogntion

Using graphs and progress bars, a dashboard shows daily activity information such as steps done, calories burnt, and active minutes.

App Name	Sensors Used	Activities Recognized	AI Features	Obesity Management Support
Google Fit [15]	Accelerometer, Gyroscope, GPS	Walking, running, cycling, standing, stairs	Heart Points system, move minutes	Tracks daily activity and offers suggestions based on intensity levels
Samsung Health [16]	Accelerometer, Gyroscope, Barometer, GPS	Walking, running, elliptical, climbing stairs, cycling	Smart goals and adaptive coaching	Tracks activity and provides insights into calorie burn
Apple Health [17]	Accelerometer, Gyroscope, GPS, Wearables	Steps, flights climbed, walking/running distance	Integrates ML for trends	Offers daily activity summaries and fitness ring goals
Fitbit App [18]	Accelerometer, Heart Rate Sensor (via device)	Running, walking, sedentary behavior, sleep	Tracks trends using ML	Offers personalized fitness plans for weight loss
Strava [19]	GPS, Accelerometer	Running, cycling, walking (outdoor activities)	Performance analytics, ML-based stats	Encourages regular exercise via social interaction
BalanceBite	Accelerometer, Gyroscope	Daily movement, inactivity, exercise detection	Real-time alerts, pattern detection	Adjusts dietary plans based on real movement data

Table 2.2 Activity Recognition Apps Comparison

2.3 Dietary Plans

Dietary programs are essential for managing health and preventing obesity. These days, AI and digital technologies enable the development of personalized food regimens based on real-time data inputs, cultural backgrounds, health issues, and personal preferences. Age, gender, body mass index (BMI), basal metabolic rate (BMR), food preferences, allergies, and pre-existing medical disorders like diabetes or hypertension are just a few of the variables that are taken into account in these customized meal plans.

2.3.1 How Dietary Plans Are Suggested:

2.3.1.1. Initial User Input:

Our system generates dietary programs using a thorough and flexible procedure that starts with the user's initial input. Users are asked for basic personal information, such as their age, height, and weight, when they initially join up. Additionally, the system collects data regarding any current medical issues, such as obesity or diabetes. Users also indicate their dietary requirements and preferences, including if they are vegetarian, non-vegetarian, or allergic to any particular food. A more individualized food recommendation is based on this core data.

2.3.1.2. AI-Based Analysis:

The system uses AI-based analysis after the initial input is gathered to gain a deeper understanding of the user's requirements and routines. Utilizing the food recognition system, the app analyzes past data pertaining to the user's past food intake in order to spot consumption trends. The system uses this information to determine the user's daily calorie and nutritional requirements. In order to create a comprehensive picture of the user's lifestyle, the system also incorporates behavior insights obtained from the activity recognition module. These patterns of behavior are crucial for matching the user's energy expenditure and physical activity to the food plan.

2.3.1.3. Plan Generation:

The system then uses this analysis to create a personalized diet plan that fits the user's objectives, be they weight loss, muscle growth, or weight maintenance. The meal plans that are produced are culturally appropriate and include readily available local foods and recipes. In order to provide suitable recommendations for breakfast, lunch, dinner, and snacks, the system additionally classifies meal recommendations according to the time of day.

2.3.1.4. Feedback Loop:

Lastly, a feedback loop is incorporated into the system to continuously enhance its suggestions. Reviewing each meal they eat and noting whether they enjoyed, didn't enjoy, or skipped it is recommended for users. Reinforcement learning techniques are then used to process this data, enabling the AI to gradually learn user preferences and adjust its nutritional recommendations accordingly. This dynamic adaptation guarantees that the system will continue to respond to each user's changing requirements and preferences.



Figure 2.3 Interface Of Dietary Plans

The picture displays an app UI that assists users in tracking their workout plan, which includes yoga, pilates, and dance activities, and eating plan, which includes meals, calories, and nutrition information. It seeks to combine fitness and nutrition management into a single platform.

Feature	**MyFitnessPal** [20]	**Foodvisor** [8]	**HealthifyMe** [20]	**CalorieMama AI** [13]	**BalanceBite**
Type of Dietary Plan	Calorie-based manual plans	Automated suggestions based on image recognition	Customized plans by nutritionists	No structured plan, only calorie estimation	AI-generated, personalized plans
Input Method	Manual logging of meals	Food photo + optional manual entry	Manual + AI logging	Food photo only	Food photo, manual entry optional
Cultural Food Inclusion	Generic international food list	Mostly Western/European foods	Indian and some Asian foods	Western-focused training data	Extensive South-Asian food database
Health Condition Support	Basic filters (e.g., diabetic, low carb)	Limited to general calorie awareness	Supports diabetic, PCOS, weight loss	Not condition-specific	Supports obesity, diabetes, and other user-defined goals
AI/ML Usage for Meal Planning	Minimal – mostly user-driven	Uses AI for recognition but limited planning	Uses AI for habit analysis and trends	AI used only for food recognition	Vision Transformer + ML for full dietary analysis
Plan Customization Level	User adjusts macro goals	AI gives macro breakdown based on photo	Human + AI assisted customization	No customization beyond food identification	Fully personalized based on user data + cultural factors
Feedback & Plan Updates	Manual tracking and adjustments	Updates via logged photos	Dietitian may revise plans	No feedback loop	Reinforcement learning adjusts plan based on feedback
Meal Recommendations	User browses recipe suggestions	Limited to calorie feedback	Based on fitness goal & food type	None	Time-based, goal-specific, and culturally relevant
Progress Tracking Tools	Graphs for calories/macros	Macronutrient distribution	Full analytics dashboard	Basic summaries	Interactive charts, goal tracking, daily reviews

Table 2.3 Dietary Plan Suggestion Comparison

REQUIREMENT SPECIFICATIONS

3.1 System Overview and Requirements

In this chapter we discuss the existing system features plus its problems and present our solution to fix them. The document provides both use case descriptions as well as functional and non-functional specifications for our proposed system. The subsequent headings are shown in the following list:

3.1.1 Use cases

3.1.2 Requirement Specifications

3.1.3 Proposed System

3.2 Existing System

A range of current mobile applications serve different purposes but display varied features to their default users.

3.2.1 MyFitnessPal

According to many users MyFitnessPal provides exceptional food and fitness tracking features. Though mainly Western-oriented it offers several major food features plus barcode reading and workout equipment interface [20].

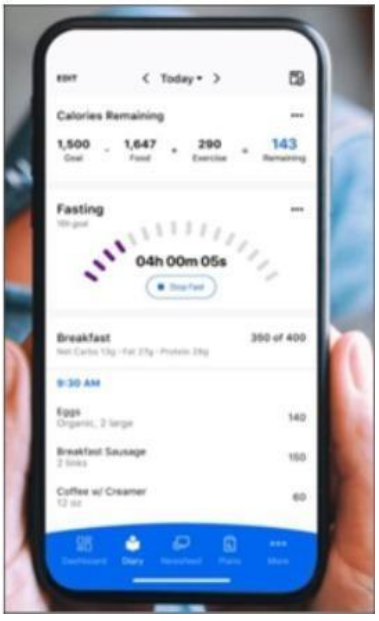


Figure 3.1 Interface of myfitnesspal

3.2.2 CalorieMama AI

The well-known CalorieMama AI system helps users determine food items and calculate their calorie intake through AI technology. This AI system analyzes pictures of food to count calories and distinguish different foods through its CNN technology. The tool performs poorly in South-Asian areas since its training data mostly features Western meals [13].



Figure 3.2 Interface of CaloriemamaAI

3.3. Limitations Of Existing Systems

3.3.1. When users need to enter their food manually the result shows unreliable nutrients and calorie counts.

3.3.2. Most apps today do not take advantage of powerful machine learning models to analyze food images and measure their dietary content.

3.3.3. The standard ideas these systems recommend don't fit individual user preferences and cultural backgrounds.

3.3.4. User interface complexity stops many programs from serving a broad range of users

3.4. Proposed Systems

The suggested BalanceBite application solves current system problems by applying Vision Transformer AI technology with machine learning to deliver better performance. The Flutter application functions across multiple platforms to offer these features to users.

3.4.1. Through Vision Transformers the system detects the food type then presents complete nutritional details for what the user captures.

3.4.2. Our app focuses on South Asian food by giving users recipe suggestions and nutrition details that match their regional diet.

3.4.3. The user-friendly design of the app uses interactive elements such as charts and progress bars alongside easy navigation which Flutter creates for better user interaction.

3.4.4. The app integrates phone movement sensors to monitor physical activities and estimate your energy burn from those movements.

3.5. Requirement Spectifications

The list sees functional and nonfunctional specifications together.

3.5.1. Functional Requirements

3.5.1.1. Food Recognition:

BalanceBite analyzes user-uploaded food photos using Vision Transformers. The program offers a thorough nutritional analysis of a food when it has been identified, including the number of calories and the values of the macronutrient's proteins, fats, and carbs. By choosing dishes from a database, users can also manually log meals, doing away with the requirement for outside assistance. The software also shows users how much food they eat on a daily and monthly basis, which helps them keep an eye on dietary trends and make wise choices.

3.5.1.2. Activity Tracking:

The program uses the accelerometers and gyroscopes that are incorporated into smartphones to measure and log physical activity in real time. It helps users maintain awareness of their physical health by tracking steps, exercise intensity, and calories burned. Consistent activity is encouraged by this automated tracking, which eliminates the need for extra wearable technology.

3.5.1.3. Progress Visualization:

The program includes aesthetically pleasing progress displays, such graphs and charts, to increase motivation. Over time, these tools assist users in observing changes in their exercise and nutrition routines. To encourage users to continue with their wellness path, the system also sends out periodic reminders and push notifications.

3.5.2. Non Functional Requirements

3.5.2.1. Performance: Low latency in picture processing and result delivery. Accurate real-time predictions for dish recognition.

3.5.2.2. Scalability: Our system delivers excellent backend operations to handle large numbers of users. Support further growth by asking the app to introduce added activities and culinary dishes.

3.5.2.3. Platform Compatibility: You should use Flutter as your development platform to achieve Android app compatibility.

3.5.2.4. Security: To keep user information safe we should use encryption to protect their data. Follow applicable data privacy rules in the area.

3.5.2.5. Usability: Build a simple interface that serves users of all age levels.

DESIGN AND METHODOLOGY

4.1 System Design

Our system is based on a modular design that combines cloud-based data management, phone sensors, machine learning, and mobile app development to provide a comprehensive solution for activity tracking and individualized nutrition. The Flutter framework, which was selected for the application's development due to its excellent performance on Android smartphones and support for interactive and aesthetically pleasing interfaces, was used. We were able to create an intuitive user interface with features like activity tracking, food detection, and customized meal suggestions thanks to Flutter's cross-platform capabilities and extensive widget library.

Each entry in the manually chosen dataset of 300 South Asian cuisines was accompanied by verified nutritional data. Our food classification model was trained on this dataset using the Vision Transformer (ViT), whose attention-based architecture makes it an excellent choice for picture classification tasks. In order to provide real-time nutritional analysis, the model was integrated into the Flutter app to detect food from user-submitted photos. We also developed a distinct collection of non-food items to increase accuracy and manage special scenarios. This guarantees that the system will recognize and notify the user if there is no food in the picture. We then added training with the YOLOv8 model for improved accuracy in non-food detection.

The app uses built-in phone sensors, such as the accelerometer and gyroscope, to track physical activity in addition to food detection. In order to communicate with these sensors, we investigated different Flutter libraries and put tracking capabilities in place to keep an eye on user movement, energy usage, and advancement over time. To display the user's progress and activity data in an easy-to-understand manner, a specific user interface screen was created. The MongoDB cloud database, which was chosen for its scalability and capacity to effectively handle sizable and unstructured datasets, houses the data collected by these sensors. In order to guarantee seamless communication between the application and the database, fetch and retrieve methods were put in place, updating user data in response to real-time sensor inputs.

We worked with a professional dietitian and a gym fitness specialist to verify the precision and efficacy of our suggestion method. The nutritionist verified the nutritional statistics and assisted in creating an organized food plan. These meal plans, which include breakfast, lunch, and dinner, are customized based on the user's BMI, exercise objectives, and nutritional preferences. In a similar vein, the fitness specialist examined our manually assembled exercise dataset to make sure the recommended activities matched user requirements and skill levels. With the help of the gathered data, the recommendation engine provides thorough health advice, dynamically updating meal and exercise recommendations in response to user engagement and data input.

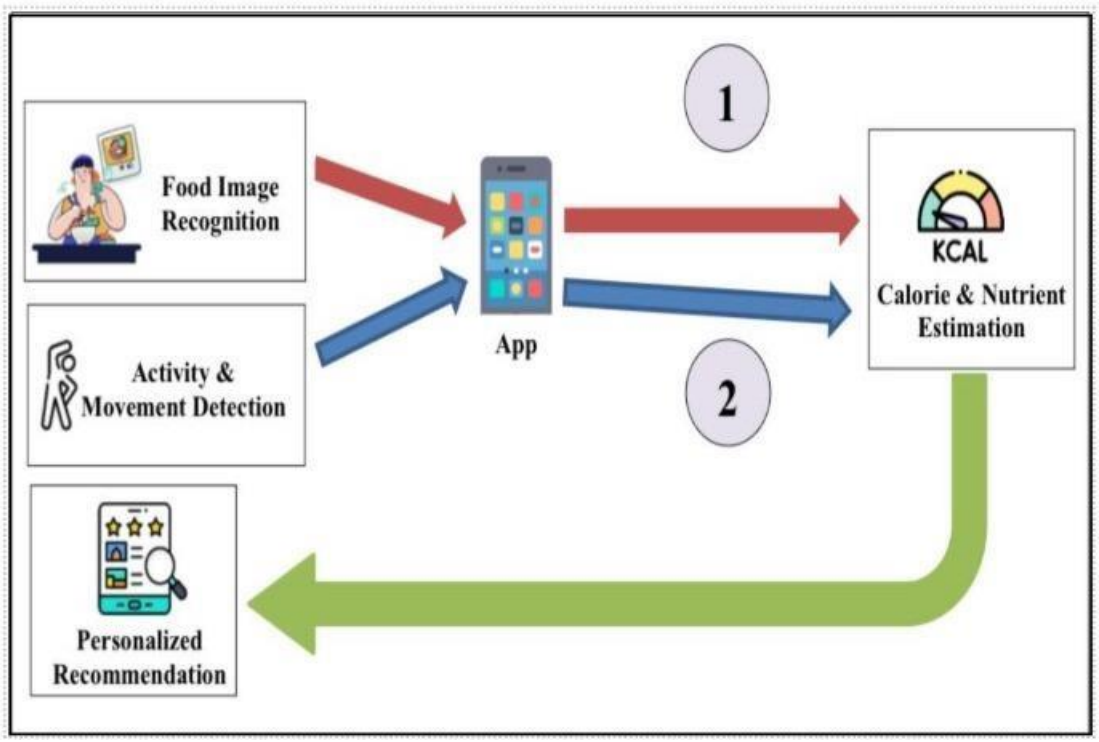


Figure 4.1 Design and Methodology

Through a client-server approach the system connects its MongoDB data center with its user interface. The Vision Transformer model tells patients precisely what foods and health advice they should follow after the system receives their data input.

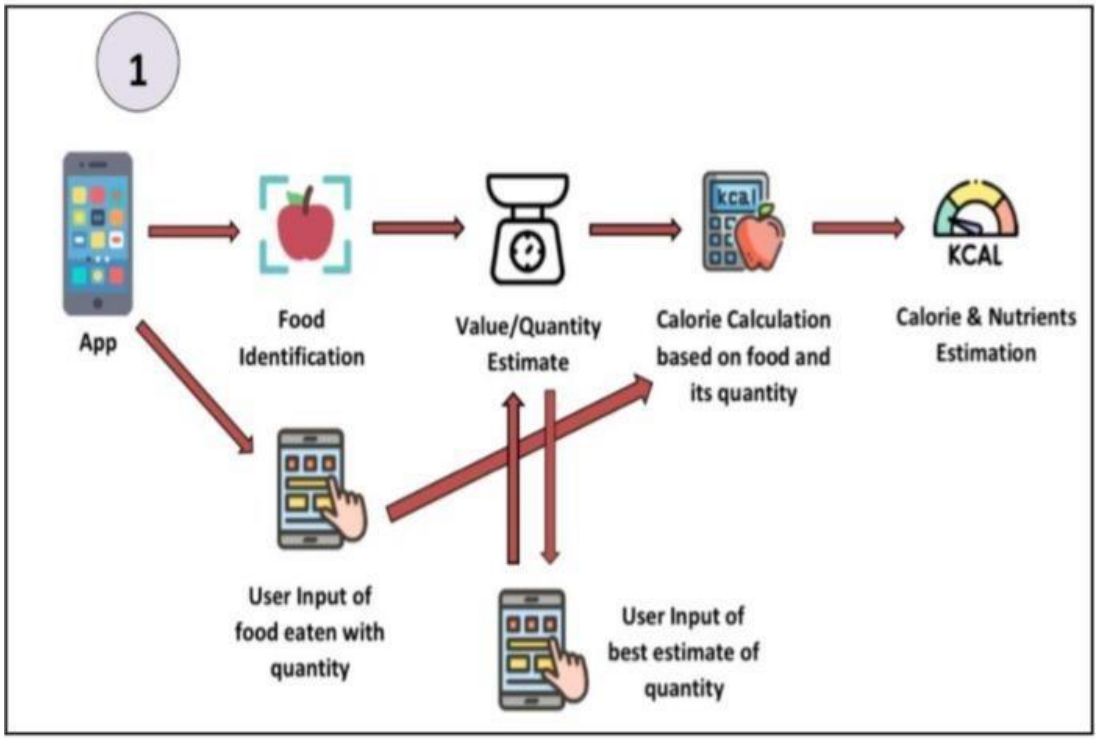


Figure 4.2 Process of Food identification and nutrition estimation

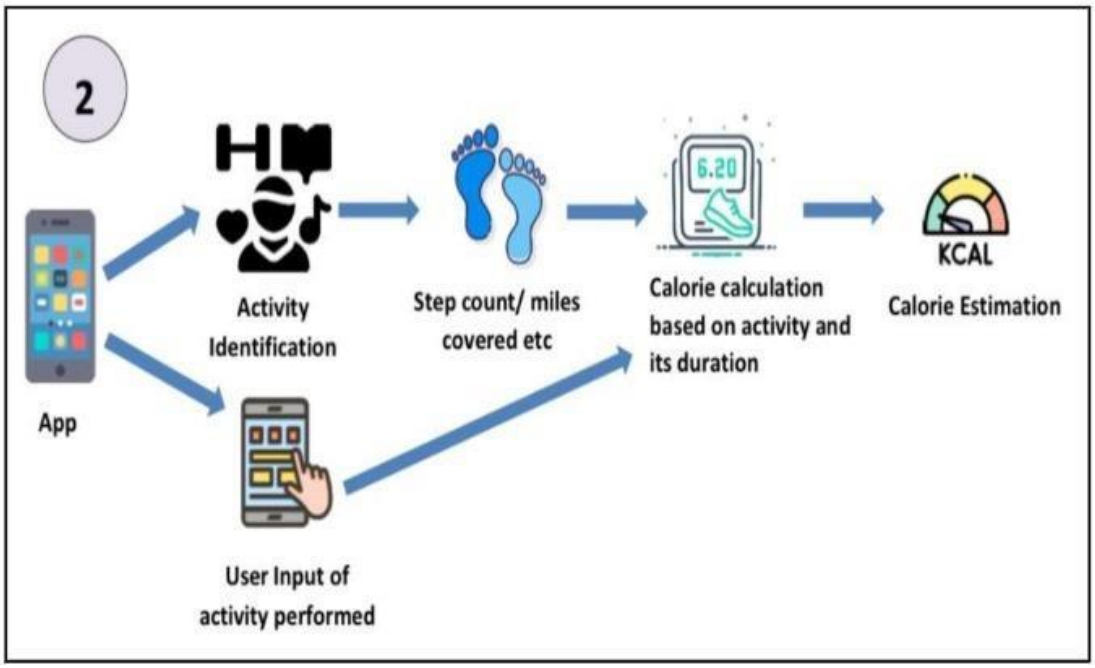


Figure 4.3 Activity identification and calorie estimation process

4.2 Dataset Preparation

In order to ensure precise food identification and nutritional estimate, the BalanceBite dataset preparation procedure is essential. In order to enhance the learning process of the Vision Transformer model and enable it to generalize across a broad range of South Asian meals, it entails gathering, organizing, improving, and structuring data.

4.2.1 Dataset Creation

BalanceBite uses a special database designed especially for South Asian food to run our application. This dataset offers comprehensive nutritional insights and was thoughtfully selected to represent the variety of regional cuisines. At the moment, the database contains nutritional data for more than 300 Asian foods, and each entry is characterized by the following crucial factors:

- **Food Name**
- **Portion Size (grams)**
- **Calories (kcal)**
- **Proteins (g)**
- **Fats (g)**
- **Net Carbohydrates (g)**

An example portion of the dataset is shown below:

Food Name	Portion Size (grams)	Calories (kcal)	Proteins (g)	Fats (g)	Net Carbs (g)
Aloo Baingan	135	95	2.2	5.1	12.7
Aloo Gobi	155	111	2.5	5.5	15
Aloo Gosht	86	118	6.76	7.6	5.95
Aloo Mattar	28.3	35.8	1.3	2.3	3
Aloo Methi	240	251	4.2	14.7	27.6
Aloo Paratha	100	220	4	9	30
Aloo Tikki	25	51	1	2	7
Aloo Tikki Chaat	180	208	4.78	10.35	27.28
Arbi Sabzi	100	110	2.13	2.94	21.13
Awadhi Chicken Biryani	240	821	40	39	73
Baida Roti	120	251	16.29	6.84	30.45
Baida Roti	120	251	16.29	6.84	30.45

Figure 4.4 Dataset (BalanceBite Dataset)

This database was prepared with special care to incorporate cuisines from various South Asian cultures and ethnic groups. In order to highlight texture, color, and surface detail essential components in image-based food recognition the model also needed clear images of every meal, taken with high-resolution cameras.

Comprehensive nutritional information, including calories and the precise amounts of macronutrients (protein, lipids, and carbs), is provided for each food entry. To guarantee accuracy and consistency in analysis, this data is selected by nutrition specialists and taken from reliable databases including USDA FoodData Central, Nutrition Facts, and validated South Asian nutrition sources [22].

4.3 Dataset Preprocessing

The photos experience diverse enhancement techniques that help the model become more universal and work better with various images. Multiple variations of real food photos make the model perform better under changing conditions. The model detects the meal subject with image segmentation procedures to separate it from other elements of the picture. The model ends up spotting dish essentials better since it ignores unnecessary background elements. The segmentation process helps the model better recognize food items through its separate analysis of dish components. The team tested several deep learning models including VGG-16, ResNet, and EfficientNetB0 at first. The tested models accomplished their goals yet failed to identify dishes successfully within this particular dataset. The Vision Transformer produced better results than other models and demonstrated its effectiveness as the most successful model [12].

Model	Accuracy	Precision	Recall	F1-Score	Strengths	Purpose	Reason For Use
EfficientNetB0	67%	0.66	0.64	0.65	Lightweight, efficient, mobile-friendly	Efficient performance with fewer resources	Good for resource efficiency, but lower accuracy
ResNet50	69%	0.68	0.66	0.67	Handles deeper networks well using residual connections	Uses residual connections for deep networks	Lower accuracy, but good for deeper architectures
VGG-16	71%	0.72	0.70	0.71	Simple architecture, good baseline performance	Simple deep learning architecture	Bette than ResNet50, but outperformed by ViT
Vision Transformer (ViT)	90%	0.91	0.89	0.90	Captures long-range dependencies and subtle image details	Attention mechanism for image analysis	Best accuracy for the dataset, handles complex features well

Table 4.1 Comparison Table

4.4 Dataset Labelling

Each food item's vital information, such as the dish's name, portion size (in grams), calorie content, and a breakdown of the macronutrients (proteins, fats, and carbs), was meticulously labeled in the dataset utilized in this study. Since they serve as the supervised learning goals, these labels are essential for training the Vision Transformer model. By directly connecting a food image to its nutritional makeup, each labeled entry aids the model in learning to correlate dietary values with visual attributes.

4.4.1 Standardization of Labeling

All entries adhered to defined naming rules and measurement units to guarantee uniformity throughout the collection. For example, energy values were provided in kilocalories (kcal) and portion sizes were consistently recorded in grams. This strategy lessened ambiguity and variability, which would have otherwise resulted in subpar learning outcomes or model confusion.

4.4.2 Cross-Verification of Nutritional Data

A number of trustworthy databases, including USDA FoodData Central and Nutritionix, were used to source and validate all nutritional values. Inaccuracies were reduced and the calorie and macronutrient data matched values established by science by cross-checking the data from several sources. This greatly improved the dataset's dependability.

4.4.3 Automation for Accuracy and Efficiency

Although the original labeling was done by hand, the validation procedure was automated with the help of specially written Python programs. These scripts verified that the numbers for carbs, lipids, protein, and calories were within the ranges predicted for normal portion sizes. The entry was marked for examination if anomalies were found, such as too many calories for a small serving. Data integrity was enhanced and human error was decreased by this automation.

4.4.4 Structured Format for Machine Learning Compatibility

The labeled data was all saved in structured formats, such CSV and JSON, so that machine learning procedures could use it. This format made the dataset scalable for future growth and enabled effective parsing during model training. Batch processing was also made possible by structured data, which sped up the incorporation of labeled entries into the training pipeline.

4.4.5 Annotation Guidelines and Visual Verification

A comprehensive set of annotation criteria was developed in order to preserve uniformity and quality. These rules made sure that every food item was appropriately labeled based on texture, color, and shape even ones with similar visual characteristics. Furthermore, labeling accuracy was verified using preliminary iterations of the model training process. The dataset was further refined by tracing misclassified samples back to their labels and fixing them.

4.5 Outcome of the Optimization Process

A high-quality labeled dataset was created with the aid of these optimization techniques: standardization, cross-verification, automation, structured formatting, and visual revalidation. The BalanceBite system's dietary detection and tracking became more reliable as a result of the Vision Transformer model's improved generalization, improved predictive performance, and decreased error rates.

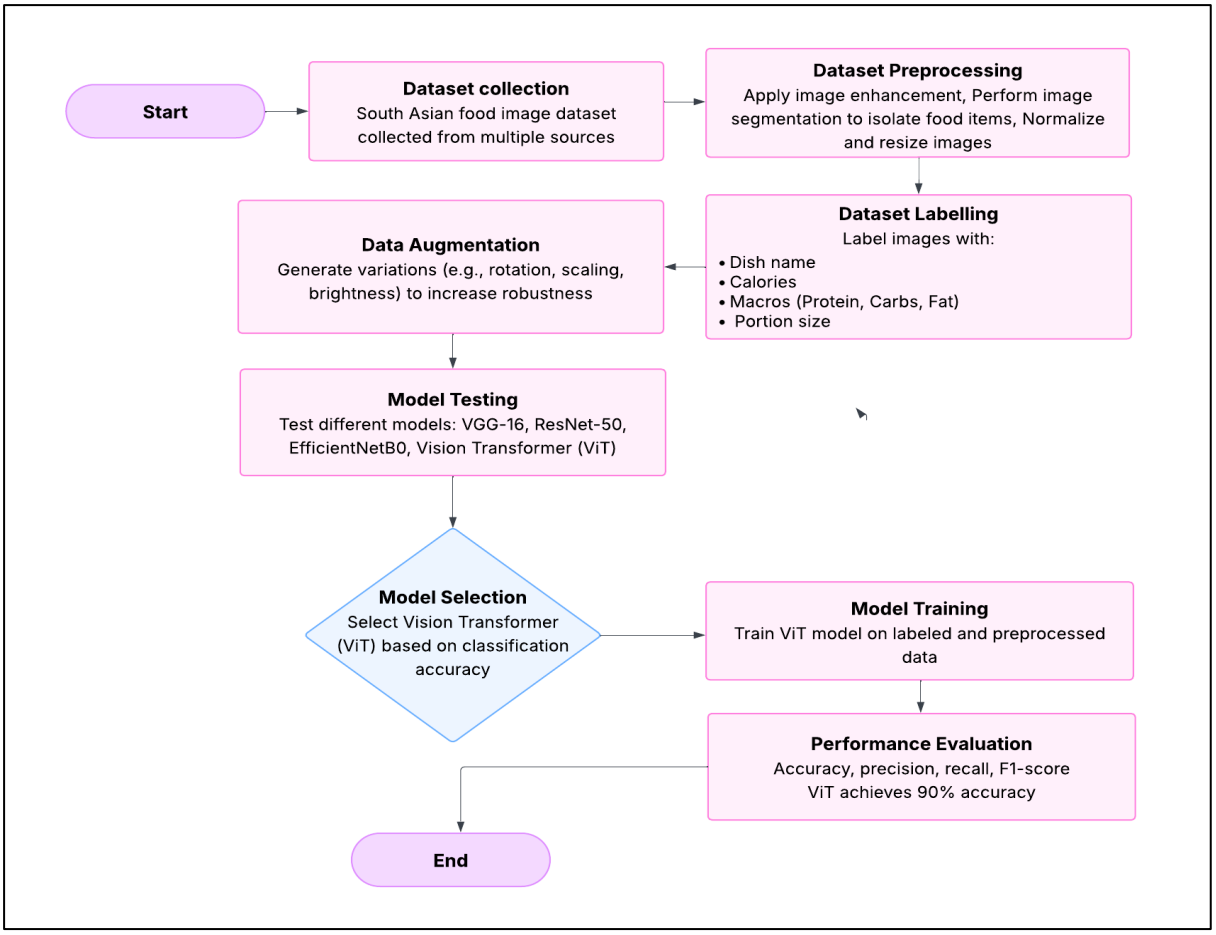


Figure 4.5: Dataset and Model Training Flowchart

4.6 CODE

4.6.1 Food Recognition:

fooddetail.dart:

```
import 'package:balancebite/dbHelper/mongodb.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'dart:io';
import 'package:image/image.dart' as img;
import 'package:onnxruntime/onnxruntime.dart';
import 'dart:typed_data';
import 'package:flutter_image_compress/flutter_image_compress.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'foodrecognition.dart';
import 'dart:typed_data';
import 'package:image/image.dart' as img;

class FoodDetail extends StatefulWidget {
  final Map<String, dynamic>? userData;
  final String imagePath;
```

```
  const FoodDetail({Key? key, required this.imagePath, required this.userData}) :
        super(key: key);

  @override
  _FoodDetailState createState() => _FoodDetailState();
}

class _FoodDetailState extends State<FoodDetail> {
  int servings = 1;
  String selectedFood = 'None';
  Map<String, dynamic> foodInfo = {
    'kcal': 0,
    'fat': 0,
    'carbs': 0,
    'protein': 0,
  };
  List<String> foodOptions = ['---'];
  bool isLoading = true;
  img.Image? fullImage;

  @override
  void initState() {
    super.initState();
    foodOptions = ['None'];
    predictFoodOptions().then((_) {
      if (foodOptions.isNotEmpty && foodOptions[0] != 'None') {
        fetchFoodInfo(foodOptions[0]);
      }
    });
  }

  Future<void> loadFullImage() async {
    try {
      final imageFile = File(widget.imagePath);
      final compressedFile = await compressImage(imageFile);
      final imageBytes = await compressedFile.readAsBytes();
      final image = img.decodeImage(imageBytes)!;
      setState(() {
        fullImage = image;
      });
    } catch (e) {
      print('Error loading full image: $e');
    }
  }

  Future<void> predictFoodOptions() async {
    try {
      final predictions = await predictFood(widget.imagePath);
      setState(() {
        foodOptions = predictions.isNotEmpty ? predictions : ['None'];
        selectedFood = foodOptions[0];
        isLoading = false;
      });
    } catch (e) {
```

```
    print('Error during prediction: $e');
    setState(() {
      isLoading = false;
    });
  }
}

Future<List<String>> predictFood(String imagePath) async {
try {
  // Load the image from the file
  final imageFile = File(imagePath);
  final imageBytes = await imageFile.readAsBytes();
  final image = img.decodeImage(imageBytes)!;

  // Preprocess the image
  final inputData = preprocessImage(image);

  // Initialize the ONNX runtime environment
  OrtEnv.instance.init();

  // Load the ONNX model from assets
  final rawAssetFile = await rootBundle.load('assets/model.onnx');
  final bytes = rawAssetFile.buffer.asUint8List();

  // Create the session for running the model
  final sessionOptions = OrtSessionOptions();
  final session = OrtSession.fromBuffer(bytes, sessionOptions);

  // Prepare the input tensor for the model
  final inputTensor = OrtValueTensor.createTensorWithDataList(
    Float32List.fromList(inputData),
    [1, 3, 224, 224], // Shape of the input tensor (batch_size, channels, height, width)
  );

  final inputs = {'input': inputTensor};
  final runOptions = OrtRunOptions();

  // Run the model to get predictions
  final outputs = await session.run(runOptions, inputs);
  final outputTensor = outputs.first!.value as List<List<double>>;
  final flattenedOutput = outputTensor.expand((element) => element).toList();

  // Log the output to debug
  print("Model output (flattened): $flattenedOutput");

  // Get top predictions from the model output
  final topPredictions = getTopPredictions(flattenedOutput, 4);

  // Release resources after running the model
  inputTensor.release();
  runOptions.release();
  for (var tensor in outputs) {
    tensor!.release();
  }
  session.release();
```

```
    OrtEnv.instance.release();

    return topPredictions;
  } catch (e) {
    print('Error during inference: $e');
    return [];
  }
}



Float32List preprocessImage(img.Image image) {
  final inputData = Float32List(224 * 224 * 3);
  int index = 0;

  // Resize the image to 224x224
  final resizedImage = img.copyResize(image, width: 224, height: 224);

  // Normalize the pixel values (mean and std values from your model)
  final mean = [0.485, 0.456, 0.406]; // ImageNet Mean (Red, Green, Blue)
  final std = [0.229, 0.224, 0.225]; // ImageNet Std (Red, Green, Blue)

  // Normalize each pixel to the range [0, 1], subtract mean, and divide by std
  for (var y = 0; y < resizedImage.height; y++) {
    for (var x = 0; x < resizedImage.width; x++) {
      final pixel = resizedImage.getPixel(x, y);
      // Normalize the RGB values (subtract mean and divide by std)
      inputData[index++] = (img.getRed(pixel) / 255.0 - mean[0]) / std[0]; // Red
          channel
      inputData[index++] = (img.getGreen(pixel) / 255.0 - mean[1]) / std[1]; // Green
          channel
      inputData[index++] = (img.getBlue(pixel) / 255.0 - mean[2]) / std[2]; // Blue
          channel
    }
  }

  return inputData;
}



List<String> getTopPredictions(List<double> outputTensor, int topN) {
  // List of food labels (same as used in training the model)
  final foodLabels = [
    'Afghani Pulao', 'Almond Milk', 'Aloo Curry', 'Aloo Gosht', 'Aloo Bukhara',
        'Alucha', 'Amlok',
    'Amrood', 'Anar', 'Apple', 'Apricot', 'Banana', 'Banana Shake', 'Beef Burger', 'Beef
        Korma',
    'Besan Ladoo', 'Black Berry', 'Carbonated Beverages', 'Chana Daal With Ghee
        Tarka', 'Chapli Kabab',
    'Cheese Ball', 'Cheese Garlic Bread', 'Chicken Burger', 'Chicken Malai Boti',
        'Chicken Nuggets',
    'Chicken Sajji With Rice', 'Chicken Shawarma', 'Chicken Strips', 'Chicken Wings',
        'Chinese Hotpot',
    'Chocolate Custard', 'Club Sandwich', 'Coconut Ladoo', 'Coffee Cappucinno', 'Coffee
        Espresso',
```

```
'Coffee Iced Latte', 'Coffee Latte', 'Custard', 'Dates', 'Doodh Jalebi', 'Doodh Patti',
'Egg Custard', 'Falsa', 'Falsa Juice', 'Fried Rice', 'Fruit Custard', 'Fruit Smoothie',
'Gosht', 'Grapes', 'Green Tea', 'Jamun', 'Kalay Chanay', 'Kashmiri Chai', 'Kinno',
'Lahori Fried Fish', 'Lasagna With Chicken', 'Lasagna With Meat', 'Lassi', 'Lemon',
'Lemon Juice', 'Loaded Fries', 'Low-Fat Custard', 'Lukhnowi Korma', 'Lychee',
    'Macaroni',
'Malta', 'Mango Juice', 'Mango Ripe', 'Milk Cake', 'Mutton Pulao', 'Naan', 'Narangi',
'Nashpati', 'Noodles', 'Papita', 'Paratha Shawarma', 'Pasta', 'Peshawari Karahi',
    'Pineapple',
'Pizza', 'Protein Shake', 'Rabri', 'Rasgulla', 'Rooh Afza', 'Rusmalai', 'Sajji', 'Sarda',
'Shikarpuri Achar', 'Sindhi Kadhi', 'Spaghetti', 'Stawberry', 'Tarbuz', 'Tikka Kabab',
'Vanilla Custard', 'Vegetable Jalfrezi', 'Zinger Burger', 'Achari Chicken', 'Akhrot',
'Aloo Capsicum', 'Aloo Chaat', 'Aloo Gawar Phali', 'Aloo Gobhi', 'Aloo Ki Bhujia',
'Aloo Matar', 'Aloo Methi', 'Aloo Mungray', 'Aloo Paratha', 'Aloo Qeema', 'Aloo
    Tikki',
'Aloo Turai', 'Anda Bhurji', 'Anda Chana', 'Anday Ka Halwa', 'Anday Wala Burger',
    'Arbi Sabzi',
'Badam', 'Baingan Ka Bharta', 'Banana Kheer', 'Barfi', 'Beef Chili Dry', 'Besan Ka
    Halwa', 'Bhel Puri',
'Bhindi', 'Bhuna Gosht', 'Bihari Kabab', 'Biryani', 'Biscuit', 'Boiled Egg', 'Bun
    Kebab', 'Butter Chicken',
'Chana Daal Halwa', 'Chana Masala', 'Chana_Puri', 'Chapati', 'Chicken Cheese
    Omelette', 'Chicken Chungay',
'Chicken Ginger Curry', 'Chicken Handi', 'Chicken Patties', 'Chicken Sandwiches',
    'Chicken Shashlik',
'Chicken Yakhni Soup', 'Chilgoza', 'Chinese Spring Rolls', 'Chow Mein', 'Crab',
    'Curry Pakora',
'Daal Chawal', 'Daal Gosht', 'Daal Mash', 'Dahi Baingan', 'Dahi Bhalla', 'Dahi Papri
    Chaat', 'Dahi Puri',
'Desi Murgh', 'Dhokla', 'Donuts', 'Double Roti', 'Dum Arbi', 'Dum Pukht',
    'Dumplings', 'Egg Fried', 'Fafda', 'Falooda', 'Feerni', 'Fish (Khaga)', 'Fish
    (Promfret)', 'Fish (Roa)', 'Fish (Shanghara)', 'Fish (Soal)', 'Fish (Surmai)', 'Fish
    Biryani', 'Fish Karahi', 'Fish Pakora', 'Fish Tikka', 'French Toast', 'Fried Aloo',
    'Fried Chicken Wings', 'Fried Fish', 'Gobi Manchurian', 'Gobi Paratha', 'Gol
    Gappay', 'Gola Kabab', 'Gur Wala Parathay', 'Haleem', 'Halwa Gajjar', 'Halwa
    Puri', 'Halwa Suji', 'Hot And Sour Soup', 'Icecream', 'Idli', 'Jalebi With Rabri',
    'Jaleebi', 'Jau Ki Roti', 'Kachori With Chutney', 'Kaddu Ka Halwa', 'Kaju',
    'Kaju Katli', 'Karahi', 'Karela Gosht', 'Kat A Kat', 'Keema', 'Keema Paratha',
    'Kheer', 'Khopra', 'Khow Suey', 'Khoya', 'Kichri', 'Kofta', 'Kulcha', 'Kulfi',
    'Kung Pao Beef', 'Lab E Shireen', 'Lachha Paratha', 'Lahori Chargha', 'Lemon
    Chicken', 'Lobia', 'Makki Roti', 'Malai Boti', 'Masoor Dal', 'Matar', 'Matar
    Paneer', 'Mathri', 'Methi Chicken', 'Mint Raita', 'Mithai', 'Mix Sabzi', 'Mooli
    Paratha', 'Muffin', 'Murgh Makhni', 'Mutton Chops Masala', 'Mutton Handi',
    'Namak Paray', 'Nihari', 'Ojri', 'Omelette', 'Pakoras', 'Palak Gosht', 'Palak
    Paneer', 'Pancakes', 'Paratha', 'Paratha Rolls', 'Paya', 'Peshawari Chapli
    Kebab', 'Pista', 'Prawn', 'Prawn Karahi', 'Pulao', 'Qorma', 'Raan Roast', 'Raita',
    'Rajma Chawal', 'Roghni Naan', 'Saag Paneera', 'Samosa', 'Samosa Chaat',
    'Sarson Ka Saag', 'Seekh Kabab', 'Seviyan', 'Shalgam Ki Sabzi', 'Shami
    Kabab', 'Sheer Khurma', 'Sheermal', 'Sindhi Biryani', 'Siri Paye', 'Sohan
    Halwa', 'Sonf', 'Soup', 'Supari', 'Sweet And Sour Chicken', 'Tandoori Chicken',
    'Tandoori Chicken Wraps', 'Tandoori Fish', 'Tandoori Roti', 'Tikka Boti',
    'Tinda Sabzi', 'Trifle', 'Upma', 'Vegetable Pakora', 'Vegetable Spring Rolls',
    'White Dalia', 'White Karahi', 'Yogurt', 'Zarda'
];
```

```
  // Sort the predictions based on the output tensor
  final sortedIndices = List<int>.generate(outputTensor.length, (i) => i)
    ..sort((a, b) => outputTensor[b].compareTo(outputTensor[a]));

  // Return the top N predictions
  return sortedIndices.take(topN).map((index) => foodLabels[index]).toList();
}

  Future<void> fetchFoodInfo(String foodName) async {
    try {
      final data = await MongoDBService.fetchFoodInfo(foodName);
      setState(() {
        foodInfo = {
          'kcal': data['kcal'],
          'fat': data['fat'],
          'carbs': data['carbs'],
          'protein': data['protein'],
        };
      });
    } catch (e) {
      print('Error fetching food info: $e');
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF212020),
      appBar: AppBar(
        backgroundColor: const Color(0xFF000000),
        title: const Text('Food Detail'),
      ),
      body: isLoading
          ? const Center(child: CircularProgressIndicator())
          : SingleChildScrollView(
              child: Padding(
                padding: const EdgeInsets.all(16.0),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Image.file(
                      File(widget.imagePath),
                      width: double.infinity,
                      height: 200,
                      fit: BoxFit.cover,
                    ),
                    const SizedBox(height: 20),
                    Text(
                      selectedFood,
                      style: const TextStyle(
                        fontSize: 24,
                        color: Colors.white,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
```

```
const SizedBox(height: 10),
Row(
  children: [
    Text(
      '${foodInfo['kcal'] * servings} KCal',
      style: const TextStyle(
        fontSize: 18,
        color: Colors.white,
      ),
    ),
    const Spacer(),
    IconButton(
      icon: const Icon(Icons.remove, color: Colors.white),
      onPressed: () {
        setState(() {
          if (servings > 1) servings--;
        });
      },
    ),
    Text(
      '$servings',
      style: const TextStyle(
        fontSize: 18,
        color: Colors.white,
      ),
    ),
    IconButton(
      icon: const Icon(Icons.add, color: Colors.white),
      onPressed: () {
        setState(() {
          servings++;
        });
      },
    ),
  ],
),
const SizedBox(height: 20),
DropdownButton<String>(
  value: selectedFood,
  items: foodOptions.map((String value) {
    return DropdownMenuItem<String>(
      value: value,
      child: Text(
        value,
        style: const TextStyle(color: Colors.white),
      ),
    );
  }).toList(),
  onChanged: (newValue) {
    setState(() {
      selectedFood = newValue!;
    });
    fetchFoodInfo(selectedFood);
  },
  dropdownColor: const Color(0xFF212020),
```

```
    ),
    const SizedBox(height: 20),
    const Text(
      'Nutritional Information',
      style: TextStyle(
        fontSize: 20,
        color: Colors.white,
        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 10),
    _buildNutritionalInfo('KCal', '${foodInfo['kcal'] * servings}'),
    _buildNutritionalInfo('Fat', '${foodInfo['fat'] * servings}g'),
    _buildNutritionalInfo('Carbs', '${foodInfo['carbs'] * servings}g'),
    _buildNutritionalInfo('Protein', '${foodInfo['protein'] * servings}g'),
    const SizedBox(height: 20),
    Center(
      child: ElevatedButton(
        onPressed: () async {
          try {
            final foodDetails = {
              'foodName': selectedFood,
              'servings': servings,
              'kcal': foodInfo['kcal'] * servings,
              'fat': foodInfo['fat'] * servings,
              'carbs': foodInfo['carbs'] * servings,
              'protein': foodInfo['protein'] * servings,
            };

            final imageFile = File(widget.imagePath);

            final userId = widget.userData?['_id'];

            if (userId == null) {
              throw Exception('User ID or email not found in userData');
            }

            await MongoDBService.addFoodToDiary(userId, foodDetails,
imageFile);

            ScaffoldMessenger.of(context).showSnackBar(
              const SnackBar(content: Text('Added to food diary')),
            );

            Navigator.pushReplacement(
              context,
              MaterialPageRoute(
                builder: (context) => FoodRecognition(userData:
widget.userData),
              ),
            );
          } catch (e) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Failed to add food to diary: $e')),
            );
```

```
                  }
                },
                style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFF74D9EA),
                  padding: const EdgeInsets.symmetric(horizontal: 40, vertical: 15),
                ),
                child: const Text(
                  'Add to Food Diary',
                  style: TextStyle(
                    fontSize: 16,
                    color: Colors.black,
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    ),
  );
}

Widget _buildNutritionalInfo(String label, String value) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 8.0),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          label,
          style: const TextStyle(
            fontSize: 16,
            color: Colors.white,
          ),
        ),
        Text(
          value,
          style: const TextStyle(
            fontSize: 16,
            color: Colors.white,
            fontWeight: FontWeight.bold,
          ),
        ),
      ],
    ),
  );
}

Future<File> compressImage(File file) async {
  final tempDir = Directory.systemTemp;
  final tempFile =
          File('${tempDir.path}/compressed_${DateTime.now().millisecondsSinceEpo
          ch}.jpg');

  final result = await FlutterImageCompress.compressAndGetFile(
```

```
    file.absolute.path,
    tempFile.absolute.path,
    quality: 50,
   );

   if (result == null) {
    throw Exception('Failed to compress image');
   }

   return File(result.path);
  }
          }
```

4.6.2 Activity Recognition

activityrecognition.dart:

```
import 'dart:async';
import 'dart:math';
import 'package:balancebite/screens/home.dart';
import 'package:flutter/material.dart';
import 'package:sensors_plus/sensors_plus.dart';
import 'package:light/light.dart';
import 'package:intl/intl.dart';
import 'package:balancebite/dbHelper/local_database.dart';
import 'package:balancebite/screens/foodrecognition.dart';
import 'package:balancebite/screens/menu.dart';
import 'package:connectivity_plus/connectivity_plus.dart';

class ActivityRecognitionScreen extends StatefulWidget {
  final Map<String, dynamic>? userData;

  const ActivityRecognitionScreen({super.key, this.userData});

  @override
  State<ActivityRecognitionScreen> createState() =>
      _ActivityRecognitionScreenState();
}

class _ActivityRecognitionScreenState extends State<ActivityRecognitionScreen> {
  List<double> _accMagnitudes = [];
  List<double> _gyroValues = [];
  double _lightLevel = 0.0;
  String _predictedActivity = 'Detecting...';
  String _lastActivity = '';
  Map<String, int> _activityDurations = {};
  Map<String, int> _activityGoals = {
    'Running': 300,
    'Walking': 600,
    'Jumping': 180,
    'Sleeping': 28800,
    'Sitting': 1200, // for example, 20 minutes
  };
```

```
// Date filtering related variables
DateTime _selectedDate = DateTime.now();
DateTime _filterStartDate = DateTime.now();
DateTime _filterEndDate = DateTime.now();
String _filterType = 'Today'; // 'Today', 'Date Range', 'All Time'

late Timer _processingTimer;
late Timer _durationTimer;
int _currentActivityDuration = 0;
DateTime? _currentActivityStartTime;
DateTime? _stillStartTime;

Light? _light;
StreamSubscription? _lightSubscription;

// Dark theme colors
final _darkBackground = Color(0xFF121212);
final _cardBackground = Color(0xFF1E1E1E);
final _surfaceColor = Color(0xFF2C2C2C);
final _primaryTextColor = Colors.white;
final _secondaryTextColor = Colors.white70;
double _userWeight = 70.0; // Default weight in kg

Map<String, dynamic>? _userData;
bool _isOnline = true;
final Connectivity _connectivity = Connectivity();

int _selectedIndex = 2; // Activity is the 3rd index (0-based)

@override
void initState() {
  super.initState();
  _userData = widget.userData;
  _checkConnectivity();
  _connectivity.onConnectivityChanged.listen((results) {
    if (results.isNotEmpty) {
      _updateConnectionStatus(results.first);
    }
  });
  _startSensorListeners();
  _processingTimer = Timer.periodic(
    Duration(seconds: 2),
    (_) => _classifyActivity(),
  );
  _durationTimer = Timer.periodic(
    Duration(seconds: 1),
    (_) => _updateCurrentActivityDuration(),
  );
}

Future<void> _checkConnectivity() async {
  final results = await _connectivity.checkConnectivity();
  if (results.isNotEmpty) {
    _updateConnectionStatus(results.first);
  }
```

```
}

void _updateConnectionStatus(ConnectivityResult result) {
  setState(() {
    _isOnline = result != ConnectivityResult.none;
  });
}

void _startSensorListeners() {
  accelerometerEvents.listen((event) {
    double mag = sqrt(
      event.x * event.x + event.y * event.y + event.z * event.z,
    );
    _accMagnitudes.add(mag);
    if (_accMagnitudes.length > 100) _accMagnitudes.removeAt(0);
  });

  gyroscopeEvents.listen((event) {
    double motion = sqrt(
      event.x * event.x + event.y * event.y + event.z * event.z,
    );
    _gyroValues.add(motion);
    if (_gyroValues.length > 100) _gyroValues.removeAt(0);
  });

  _initializeLightSensor();
}

void _initializeLightSensor() {
  try {
    _light = Light();
    _lightSubscription = _light?.lightSensorStream.listen((int luxValue) {
      setState(() {
        _lightLevel = luxValue.toDouble();
      });
    });
  } catch (e) {
    print('Error initializing light sensor: $e');
    setState(() {
      _lightLevel = -1.0;
    });
  }
}

void _updateCurrentActivityDuration() {
  if (_currentActivityStartTime != null) {
    setState(() {
      _currentActivityDuration =
          DateTime.now().difference(_currentActivityStartTime!).inSeconds;
    });
  }
}

void _classifyActivity() {
  if (_accMagnitudes.length < 10 || _gyroValues.length < 10) return;
```

```
    double avgAcc =
        _accMagnitudes.reduce((a, b) => a + b) / _accMagnitudes.length;
    double varAcc =
        _accMagnitudes
            .map((e) => pow(e - avgAcc, 2).toDouble())
            .reduce((a, b) => a + b) /
        _accMagnitudes.length;
    double avgGyro = _gyroValues.reduce((a, b) => a + b) / _gyroValues.length;

    String activity;
    bool isStill = varAcc < 0.01 && avgGyro < 0.3;

    if (isStill) {
      _stillStartTime ??= DateTime.now();
      int stillDuration = DateTime.now().difference(_stillStartTime!).inSeconds;

      if (_lightLevel < 10 && stillDuration > 60) {
        activity = 'Sleeping';
      } else {
        activity = 'Sitting';
      }
    } else {
      _stillStartTime = null;

      if (avgAcc > 18 && varAcc > 8.0 && avgGyro > 0.5) {
        activity = 'Jumping';
      } else if (avgAcc > 13 && avgGyro > 1.5 && varAcc > 3.0 && varAcc < 8.0) {
        activity = 'Running';
      } else if (avgAcc > 9.5 && varAcc > 1.0 && avgAcc < 13) {
        activity = 'Walking';
      } else {
        activity = 'Sitting';
      }
    }

    setState(() {
      _predictedActivity = activity;
    });
  }

  void _updateProfile(Map<String, dynamic> updatedData) {
  setState(() {
    _userData = updatedData;
  });

  // Assuming that LocalDatabase.updateUser is the function to update user info in the
          local database
  if (updatedData['_id'] != null) {
    // Pass both the original data (userData) and updated data (updatedData)
    LocalDatabase.updateUser(_userData!, updatedData);  // <-- Pass both _userData
          and updatedData
  }
}
```

```
@override
void dispose() {
  _processingTimer.cancel();
  _durationTimer.cancel();
  _lightSubscription?.cancel();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final screenWidth = MediaQuery.of(context).size.width;

  return Scaffold(
    backgroundColor: const Color(0xFF212020),
    drawer: MenuDrawer(userData: _userData, onProfileUpdated: _updateProfile),
    body: Column(
      children: [
        // Header Section
        Container(
          color: Colors.black,
          padding: const EdgeInsets.all(16),
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              IconButton(
                icon: Image.asset(
                  'assets/Menu.png',
                  width: 24,
                  height: 24,
                  color: Colors.white,
                ),
                onPressed: () {
                  Scaffold.of(context).openDrawer();
                },
              ),
              Image.asset(
                'assets/logobd.png',
                width: 120,
                semanticLabel: 'Nutrition Tracker Logo',
              ),
            ],
          ),
        ),
        const SizedBox(height: 16),

        // Activity Content Section
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                // Your Activity Data Widgets here
              ],
```

```
            ),
          ),
        ),
      ],
    ),
    bottomNavigationBar: Theme(
      data: Theme.of(context).copyWith(
        canvasColor: const Color(0xFF000000),
      ),
      child: BottomNavigationBar(
        currentIndex: _selectedIndex,
        selectedItemColor: const Color(0xFF74D9EA),
        unselectedItemColor: Colors.white54,
        onTap: (index) {
          setState(() {
            _selectedIndex = index;
          });
        },
        items: const [
          BottomNavigationBarItem(
            icon: ImageIcon(AssetImage('assets/home.png')),
            activeIcon: ImageIcon(
              AssetImage('assets/home.png'),
              color: Color(0xFF74D9EA),
            ),
            label: '',
          ),
          BottomNavigationBarItem(
            icon: ImageIcon(AssetImage('assets/food.png')),
            activeIcon: ImageIcon(AssetImage('assets/food.png')),
            label: '',
          ),
          BottomNavigationBarItem(
            icon: ImageIcon(AssetImage('assets/activity.png')),
            activeIcon: ImageIcon(
              AssetImage('assets/activity.png'),
              color: Color(0xFF74D9EA),
            ),
            label: '',
          ),
          BottomNavigationBarItem(
            icon: ImageIcon(AssetImage('assets/recommendation.png')),
            activeIcon: ImageIcon(
              AssetImage('assets/recommendation.png'),
              color: Color(0xFF74D9EA),
            ),
            label: '',
          ),
        ],
      ),
    ),
  );
}
          }
```

SYSTEM IMPLEMENTATION

5.1 Architecture Design

The BalanceBite application designers have created a system architecture that serves both users well and works as expected. Different Android users can use the app because its development with Flutter made cross-platform mobile apps easier to build. The app uses the Vision Transformer (ViT) model to estimate South Asian food based on submitted user photos. The system back end uses MongoDB as its NoSQL database which stores both nutritional details and image inputs of dishes. The HTTP package in Dart links the database to the Flutter application so they can exchange information effectively [23].

5.2 Frontend Implementation

The BalanceBite app uses Flutter as its platform development toolkit to build the application frontend. The application works on all Android phones because it uses responsive design. The app lets users check all tracking options on the home screen through its straightforward user interface. Interactive displays visualize how users reach their health aims and achieve better physical activity plus eat less. The Flutter Charts package works as a display platform to show users the dynamic charts they create about their health path. The app enables users to capture meal images through its built-in camera technology. After taking photos ViT analyzes them to determine the dish types along with calorie counts and suitable meal suggestions [24].

5.3 Backend Implementation

The backend of BalanceBite uses MongoDB as its NoSQL database to organize and retrieve dish data with photos and nutrition details. Our system stores essential information about every dish like its label, ingredients, nutrition data including calories and macronutrients along with regional differences.

The decision to use MongoDB came from its ability to deal with large datasets and expand as needed. Data communication between these systems runs through the database connections the Flutter application provides. The system delivers secure results from data storage and retrieval because it was created with Dart.

5.4 Vision Transformer Model

The Vision Transformer (ViT) model serves as the primary component to identify dishes within BalanceBite application. Our system receives training from a special dataset made of 300 South Asian food items and their essential photos plus nutritional stats. The algorithm receives photos of dishes and studies their nutritional values through both processing and expanded datasets before making predictions. The Vision Transformer model takes the user picture as input to predict the dish. Our system delivers recommended meals for proper nutrition combined with nutritional value details. This feature runs in the application backbone to let the model and UI smoothly exchange data [26].

Parameter	Value
Number of Dishes	300
Dataset Size (Images)	300 images
Nutritional Information per Dish	1 set (per dish)
Preprocessing Steps	2 (Image preprocessing + Augmentation)
Model	Vision Transformer (ViT)
Input Format	Image (photo of dish)
Output Format	Dish prediction + Nutritional info (calories, macronutrients)
Processing Time (Typical)	Not provided (depends on hardware and implementation)
Backend Communication	Continuous (between UI & model)

Table 5.1 Vision Transformer Table

SYSTEM TESTING AND EVALUATION

6.1 Testing Methodology

Our assessment ensures BalanceBite works properly and meets its set requirements. Several basic and advanced tests took place in this phase including unit testing, integration testing, model testing and user acceptance testing (UAT). Our tests checked how the application works both in its user interface and system.

6.2 Performance Evaluation

Tests confirmed the application's performance capability combined with usability checks. Our team measured these main performance elements:

6.2.1. App Loading and Initialization Time

While the app loads user data, system setups, and necessary interface components, users must wait a little while for it to initialize. Mid-range devices had an average loading time of less than 3 seconds throughout testing, which satisfies user criteria. Lazy loading and background prefetching are two optimizations that have been used to further cut down on initial load time and prevent delays in accessing essential features like activity monitoring and meal reporting.

6.2.2. Real-Time Meal Data Entry Speed

When users entered meal information, the program showed excellent responsiveness, especially when AI-based food recognition using image capture was used. On average, the app took less than two seconds to process and offer nutritional estimates after a food image was taken. The effectiveness of the app's lightweight picture categorization model is validated by this performance. The prediction model's ability to function well in a variety of backgrounds, food varieties, and lighting situations was confirmed by ongoing testing.

6.2.3. Battery Consumption and Sensor Usage Impact

Battery performance can be greatly impacted by sensor-based features, especially those that use the accelerometer, gyroscope, and magnetometer to detect activity. Nevertheless, BalanceBite is designed to only turn on sensors when necessary for tracking, as while exercising or walking. Battery use increased by only 4–6% after a one-hour tracking session, which is negligible when compared to other similar health tracking apps. Battery drain has been minimized while preserving functionality thanks to power-saving modes and background processing modifications.

6.2.4. Network Efficiency for Meal Data Transmission

In order to transmit and receive meal data from its remote server for customized dietary advice, the application needs a steady internet connection. Both Wi-Fi and mobile data settings were tested. Even with 3G, the results demonstrated effective data interchange, with average upload/download response times of less than 1.5 seconds. In places with poor connectivity, performance is improved and bandwidth consumption is reduced by using compressed picture formats and lightweight data packets.

According to the positive results of the performance evaluation the app provided fast speed and only minimal battery usage impact.

6.3 Usability Testing

We tested the BalanceBite application user experience to provide a friendly interface for users. The usability tests assessed the design of the app screens and how well users could understand the application directions. The performance evaluation focused especially on testing key aspects. The evaluation team checked how well the features worked together by looking at their placement, color choices, and screen design. Users agree that the app provides an attractive interface that is effortless to operate.

Our studies focused on testing how well users could use the monitoring activities plus food logging with recommended diet suggestions. Users find this program simple to operate while stating that it has basic functions to track food intake and physical activity.

6.4 Accuracy and Reliability

A set of sample photos representing plates entered the testing process as input to evaluate the Vision Transformer model within the application. The model evaluation process was conducted by analyzing its computed results against genuine dish identification.

The ViT model correctly identified real dishes during more than 90% of the prediction attempts.

CONCLUSION

7.1 Conclusion

The application is a significant milestone in South Asia's use of technology to combat obesity and promote healthier lifestyles. By combining culturally relevant meal planning with artificial intelligence, specifically the Vision Transformer (ViT) model, the app effectively addresses the unique dietary habits and health concerns of users in the region. Users can make informed decisions about their diet and level of fitness thanks to its ability to track physical activity and provide personalised health advice.

Through extensive testing and evaluation, including functional, performance, usability, and accuracy assessments, the software has demonstrated impressive results in terms of dependability, efficiency, and user satisfaction. The system's ability to recognise foods, determine their nutritional content, and generate personalised meal plans ensures that users can follow sane and culturally appropriate health recommendations. Despite the app's efficiency, user feedback brings up areas for improvement, such widening the dataset to include additional regional cuisines and increasing personalized meal recommendations. It will continue to evolve and adjust to the needs of a diverse user base thanks to these impending improvements.

7.2 Future Work

Even with the advancements in managing obesity through AI-powered tools and mobile health apps, there are still a number of untapped possibilities and gaps that need to be investigated further. To provide a more comprehensive picture of user health, one important area is the integration of real-time multimodal data, which combines physiological data (such as heart rate and sleep patterns), sensor-based activity tracking, and dietary intake monitoring using food identification.

The following improvements could be made to the BalanceBite app in the future:

7.2.1. Emotion and Stress Detection:

According to new research, eating patterns and physical activity are directly impacted by mental well-being. In order to modify its recommendations appropriately, BalanceBite may integrate emotion recognition in future iterations using behavioral patterns, voice tones, and facial expression analysis. For example, rather of suggesting strenuous exercise during times of stress or depression, the app may suggest lighter, mood-enhancing foods and calming physical activities like yoga or walks.

7.2.2. Voice-Based and Multilingual Interaction:

Dietary logging will be quicker, easier, and more accessible for users with low literacy or visual impairments thanks to the addition of natural language processing (NLP), which will enable voice instructions. In addition to expanding the application's reach and popularity throughout South Asia, supporting regional languages (such as Bengali, Hindi, Punjabi, and Urdu) would guarantee that a variety of demographics could use it with ease.

7.2.3. Enhanced Cultural Food Recognition and Databases

The inaccurate portrayal of South Asian meals is a major drawback of many diet applications available today. Building a sizable, annotated image database of culturally significant foods, such as home-cooked meals and regional delicacies, may be the main goal of future BalanceBite updates. This will help people follow dietary guidelines without sacrificing their cultural identity by increasing recognition accuracy and enabling customized dietary input based on traditional meals.

7.2.4. Context-Aware and Adaptive Dietary Planning:

Future AI models could provide fully adaptive and customized dietary plans by learning from a user's daily routines, past meal choices, activity levels, allergies, religious preferences (e.g., vegetarian, Halal), and financial constraints rather than depending on generic meal plans or static calorie targets. Additionally, since eating habits usually change during holidays, fasting times, or travel, these plans might automatically adapt.

7.2.5. Gamification and Social Motivation

Health behavior modification can be made more interesting by incorporating gamified components like leaderboard rankings, virtual awards, daily challenges, and streaks. Future iterations may include social networking capabilities that allow users to push friends to reach particular health objectives, create groups, exchange accomplishments, and join cultural fitness communities. It has been demonstrated that social involvement boosts motivation and sustained adherence to behavior changes related to health.

7.2.6. Offline Access and Secure Data Management

Internet access is erratic for many South Asian consumers. A BalanceBite app that is ready for the future should be able to function offline, storing information locally and synchronizing when internet connectivity is restored. Gaining user trust while handling sensitive health data also requires integrating improved data privacy and encryption techniques that adhere to local data protection regulations.

References

[1] WHO, "World Health Organization," 7 May 2025. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight. [Accessed 8 May 2025].

[2] "International Journal of Obesity," 17 November 2008. [Online]. Available: https://www.nature.com/articles/ijo2008186. [Accessed 8 May 2025].

[3] "ScienceDirect," 6 May-June 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0033062014000279?via%3Dihub. [Accessed 8 May 2025].

[4] "Obesity," 6 September 2012. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1038/oby.2009.265. [Accessed 8 May 2025].

[5] "Food and Agriculture Organization of the United Nations," 2013. [Online]. Available: https://www.fao.org/nutrition/education/en/. [Accessed 8 May 2025].

[6] "ScienceDirect," 21-27 July 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140673612607351?via%3Dihub. [Accessed 8 May 2025].

[7] "PLOS Medicine," 15 January 2013. [Online]. Available: https://journals.plos.org/plosmedicine/article?id=10.1371/journal.pmed.1001362. [Accessed 8 May 2025].

[8] Foodvisor, "Foodvisor: Your Nutriton Assistant.," Foodvisor, 2023. [Online]. Available: https://www.foodvisor.io/en/. [Accessed 1 9 2024].

[9] "Healthify," [Online]. Available: https://www.healthifyme.com/in/. [Accessed 8 May 2025].

[10] "ResearchGate," October 2023. [Online]. Available: https://www.researchgate.net/publication/376302416_Convolutional_Neural_Network_CNN_EfficientNet-B0_Model_Architecture_for_Paddy_Diseases_Classification. [Accessed 8 May 2025].

[11] "ResearchGate," October 2019. [Online]. Available: https://www.researchgate.net/publication/337105858_Transfer_learning_using_VGG-16_with_Deep_Convolutional_Neural_Network_for_Classifying_Images. [Accessed 8 May 2025].

[12] " Food Recognition 2022," 2022. [Online]. Available: https://www.kaggle.com/datasets/awsaf49/food-recognition-2022-dataset. [Accessed 8 May 2025].

[13] "Calorie MAMA," Azumio Inc, 2017. [Online]. Available: https://caloriemama.ai/. [Accessed 9 9 2024].

[14] L. It!, "Weight Loss That Fits.," FitNow, Inc., 2023. [Online]. Available: https://www.loseit.com/. [Accessed 1 9 2024].

[15] "Google Fit," [Online]. Available: https://support.google.com/fit/#topic=6075053.

[16] "Samsung," [Online]. Available: https://www.samsung.com/pk/.

[17] "Apple Health," [Online]. Available: https://www.apple.com/health/.

[18] "Fitbit App," [Online]. Available: https://store.google.com/us/magazine/fitbit_app?hl=en-US.

[19] "Strava," [Online]. Available: https://www.strava.com/.

[20] "Myfitnesspal," [Online]. Available: https://www.myfitnesspal.com/.

[21] "Healthify," HealthifyMe Wellness Private Limited, [Online]. Available: https://www.healthifyme.com/in/. [Accessed 9 9 2024].

[22] "Nutrition Facts," [Online]. Available: https://play.google.com/store/apps/details?id=com.nutrition.facts&hl=en&pli=1.

[23] "Food Tracker," [Online]. Available: https://arxiv.org/pdf/1909.05994.

[24] "Diet Plan application using Flutter," [Online]. Available: https://flutterawesome.com/diet-plan-application-using-flutter/?utm_source=chatgpt.com#google_vignette.

[25] "Food and Agriculture Organization of the United Nations," [Online]. Available: https://www.fao.org/nutrition/education/en/.

APPENDICES

APPENDIX A: Approcal Certificate



MEAL PLAN DATABASE APPROVAL CERTIFICATE

BalanceBite Application

I, __Dr. Nimrah Chatha_____, a licensed nutritionist/dietitian, hereby confirm that I have thoroughly reviewed the meal plan recommendations and the associated database used in the **BalanceBite** application.

The meal plans and nutritional information provided within the app have been evaluated and found to be in line with current dietary guidelines and best practices for general health and wellness.

Approval Details:

App Name: BalanceBite

Database Information: The app's database includes meal plans, ingredients, calorie content, and nutritional information.

Review Scope: The database was reviewed for accuracy, completeness, and nutritional correctness based on professional standards.

Date of Review: __5-May-2025 _____

Institution/Organization Name: _____The University of Lahore_____

Signature _____Nimrah Chatha _____

Contact Information:
nimrahchattha23@gmail.com

Figure 7.1 Approval for Meal Plan By Nutritionist